

Min-Max Score Normalization, Worked Out by Hand

Before you can add a BM25 score to a cosine score, you must put them on the same scale — min-max maps any set of scores onto $[0, 1]$. Computed on real retriever outputs, with the failure mode that makes it fragile.

What this document does. It takes raw BM25 scores (range 0 to ~ 8) and dense cosine scores (range 0.4 to 0.82) for the same five documents, rescales each list to $[0, 1]$ by hand, and shows why this is the prerequisite for any weighted fusion (#20). It then exposes min-max’s Achilles heel — sensitivity to outliers and per-query rescaling — which motivates rank-based fusion (#21). Numbers from Appendix A.

Leads into: convex combination (#20) uses these normalised scores; RRF (#21) avoids them entirely.

1 The problem: incomparable scales

Hybrid search runs two retrievers and must merge their verdicts. But their scores live in different worlds: BM25 is an unbounded sum of term weights (here 0 to 8.2); cosine similarity is bounded in $[-1, 1]$ (here 0.40 to 0.82). Adding $8.2 + 0.61$ is meaningless — BM25 would swamp cosine purely because its numbers are bigger. We must first map both onto a common scale.

Intuition

Min-max is the “stretch to fit” rescaling. Find the smallest and largest score in a list, call them 0 and 1, and place everything else proportionally between. The best result always becomes exactly 1, the worst exactly 0, and the *shape* of the gaps in between is preserved. It is the simplest way to make a 0-to-8 scale and a 0.4-to-0.8 scale speak the same language.

2 The formula

$$\tilde{s}_i = \frac{s_i - \min_j s_j}{\max_j s_j - \min_j s_j} \quad \Rightarrow \quad \tilde{s} \in [0, 1], \quad \tilde{s}_{\min} = 0, \quad \tilde{s}_{\max} = 1.$$

Subtract the minimum (shift the floor to 0), divide by the range (scale the ceiling to 1). Order is perfectly preserved — it is a positive affine map — so normalisation never re-ranks a single list; it only makes two lists *addable*.

3 Worked by hand

Five documents, scored by each retriever:

	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>
BM25 <i>s</i>	8.2	3.1	6.5	0.0	1.7
dense <i>s</i>	0.61	0.82	0.55	0.79	0.40

Normalise BM25. $\min = 0.0$ (D), $\max = 8.2$ (A), $\text{range} = 8.2$. E.g. for C: $\tilde{s}_C = (6.5 - 0)/8.2 = 0.7927$.

$$\tilde{s}^{\text{BM25}} = (A\ 1.0000, B\ 0.3780, C\ 0.7927, D\ 0.0000, E\ 0.2073).$$

Normalise dense. $\min = 0.40$ (E), $\max = 0.82$ (B), $\text{range} = 0.42$. E.g. for A: $\tilde{s}_A = (0.61 - 0.40)/0.42 = 0.5000$.

$$\tilde{s}^{\text{dense}} = (A\ 0.5000, B\ 1.0000, C\ 0.3571, D\ 0.9286, E\ 0.0000).$$

Now both lists run 0 to 1: BM25's champion is *A*, dense's champion is *B*, and the two are finally on terms where they can be combined (#20).

Mini-example · order within a list is untouched

BM25 raw order is $A(8.2) > C(6.5) > B(3.1) > E(1.7) > D(0)$. After normalisation: $A(1.0) > C(0.79) > B(0.38) > E(0.21) > D(0)$ — *identical* order. Min-max only relabels the axis; it cannot promote or demote anything *within* one retriever's list. Re-ranking only happens later, when two normalised lists are mixed.

4 The failure mode

Min-max is brittle because min and max are single points:

- **Outlier sensitivity.** One freakishly high BM25 score stretches the range so that all other documents get squashed toward 0 — their real differences erased by a single outlier setting the ceiling.
- **Per-query instability.** The min and max are recomputed for *each query's* candidate set, so the same raw score of 6.5 maps to different normalised values on different queries. The scale is not absolute.
- **Tail compression.** If most scores cluster low with one high, nearly everything maps near 0 and distinctions vanish.

These are exactly the weaknesses that motivate **rank-based** fusion (RRF, #21), which throws away the fragile score magnitudes and keeps only the robust ordering. When you *do* use min-max, consider clipping outliers or using more robust statistics (e.g. percentiles) for the bounds.

Equation cheat-sheet

$$\tilde{s}_i = \frac{s_i - \min_j s_j}{\max_j s_j - \min_j s_j} \in [0, 1], \quad \tilde{s}_{\min} = 0, \quad \tilde{s}_{\max} = 1$$

order-preserving (never re-ranks one list); fragile to outliers & per-query bounds

Appendix A · Reference implementation

```
import numpy as np
def minmax(scores):
    # dict name -> raw score
    v = np.array(list(scores.values()), float)
    lo, hi = v.min(), v.max()
    return {k: round((s - lo) / (hi - lo), 4) for k, s in scores.items()}

minmax({"A":8.2, "B":3.1, "C":6.5, "D":0.0, "E":1.7})
# A 1.0 B 0.378 C 0.7927 D 0.0 E 0.2073
minmax({"A":0.61, "B":0.82, "C":0.55, "D":0.79, "E":0.40})
# A 0.5 B 1.0 C 0.3571 D 0.9286 E 0.0
```

Internal learning material. Numbers reproducible from the appendix code. v1.0